[Lecture Note] Repetition Structures

MGS3001 Python Programming for Business, Spring 2025

Chad (Chungil Chae)

Thu, 13 March 2025

Learning Objectives

- Introduction to Repetition Structures
- The while Loop: A Condition-Controlled Loop
- The for Loop: A Count-Controlled Loop
- Calculating a Running Total
- Sentinels
- Input Validation Loops
- Nested Loops
- Turtle Graphics: Using Loops to Draw Designs

SHORT VIDEO INTRODUCTION

https://www.youtube.com/watch?v=1_eMUHapCAI

Introduction to Repetition Structures

A repetition structure causes a statement or set of statements to execute repeatedly

---

```
# Get a salesperson's sales and commission rate.
sales = float(input('Enter the amount of sales: '))
comm_rate = float(input('Enter the commission rate: '))
# Calculate the commission.
commission = sales * comm_rate
# Display the commission.
print('The commission is $', format(commission, ',.2f'), sep='')
```

```
# Get another salesperson's sales and commission rate.
sales = float(input('Enter the amount of sales: '))
comm_rate = float(input('Enter the commission rate: '))
# Calculate the commission.
commission = sales * comm_rate
# Display the commission.
print('The commission is $', format(commission, ',.2f'), sep='')
# Get another salesperson's sales and commission rate.
sales = float(input('Enter the amount of sales: '))
comm_rate = float(input('Enter the commission rate: '))
# Calculate the commission.
commission = sales * comm_rate
# Display the commission.
print('The commission is $', format(commission, ',.2f'), sep='')
# And this code goes on and on ...
```

- Programmers commonly have to write code that performs the same task over and over.
- For example, suppose you have been asked to write a program that calculates a 10 percent sales commission for several salespeople.
- Although it would not be a good design, one approach would be to write the code to calculate one salesperson's commission, and then repeat that code for each salesperson.

One long sequence structure containing a lot of duplicated code.

- Disadvantages to this approach
  - The duplicated code makes the program large.
  - Writing a long sequence of statements can be time consuming.
  - If part of the duplicated code has to be corrected or changed, then the correction or change has to be done many times.
- Instead of writing the same sequence of statements over and over
  - A better way to repeatedly perform an operation is to write the code for the operation once
  - then place that code in a structure that makes the computer repeat it as many times as necessary.

This can be done with a repetition structure, which is more commonly known as a loop.

Condition-Controlled and Count-Controlled Loops

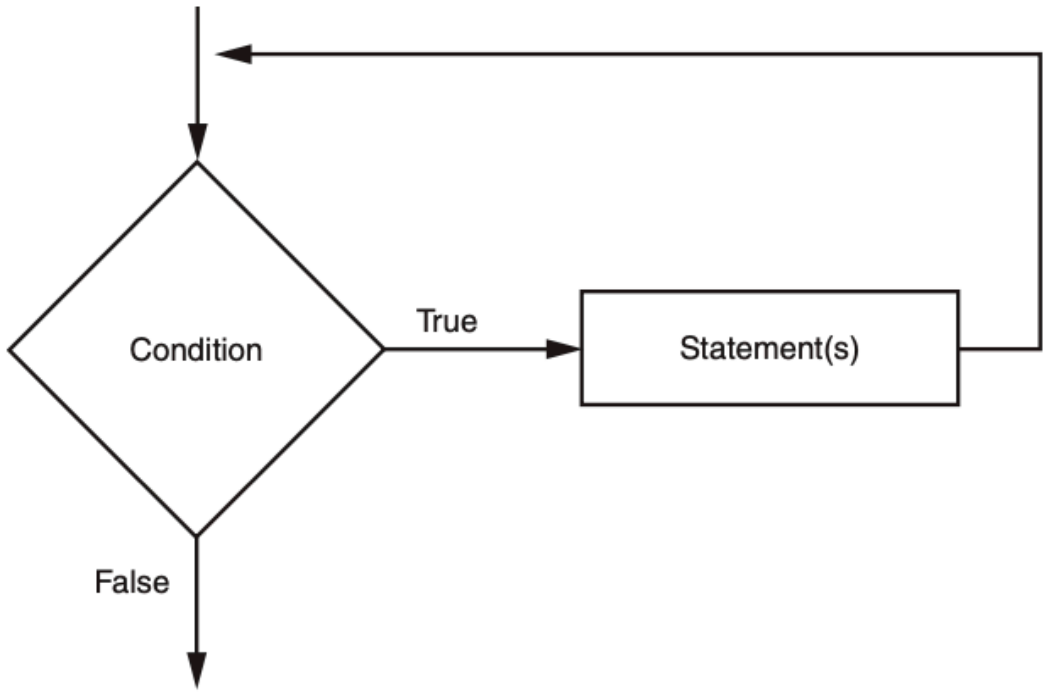
- Two broad categories of loops: **condition-controlled** and **count-controlled**.
  - **A condition-controlled loop** uses a true/false condition to control the number of times that it repeats.
  - **A count-controlled loop** repeats a specific number of times.

> **i** while statement
>
> In Python, you use the while statement to write a condition-controlled loop, and you use the for statement to write a count-controlled loop.

The while Loop: A Condition-Controlled Loop

A condition-controlled loop causes a statement or set of statements to repeat as long as a condition is true. In Python, you use the while statement to write a condition-controlled loop.

- The while loop gets its name from the way it works: while a condition is true, do some task. The loop has two parts:
  - (1) a condition that is tested for a true or false value, and
  - (2) a statement or set of statements that is repeated as long as the condition is true.

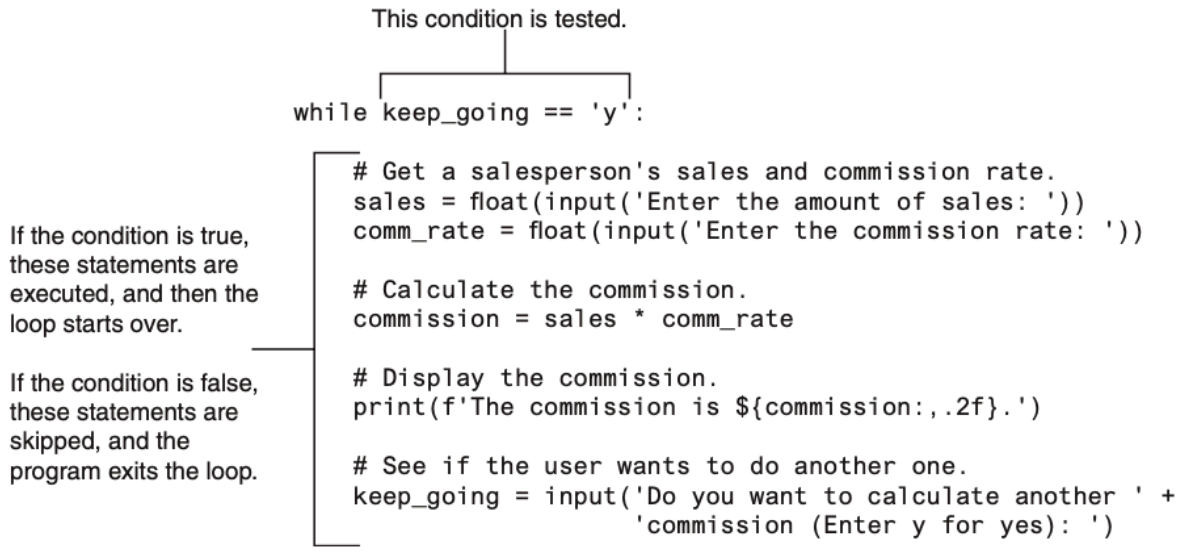


- Notice what happens if
  - the condition is true:
    - one or more statements are executed, and the program's execution flows back to the point just above the diamond symbol.

  - The condition is tested again, and if it is true,
    - the process repeats.
  - If the condition is false,
    - the program exits the loop.

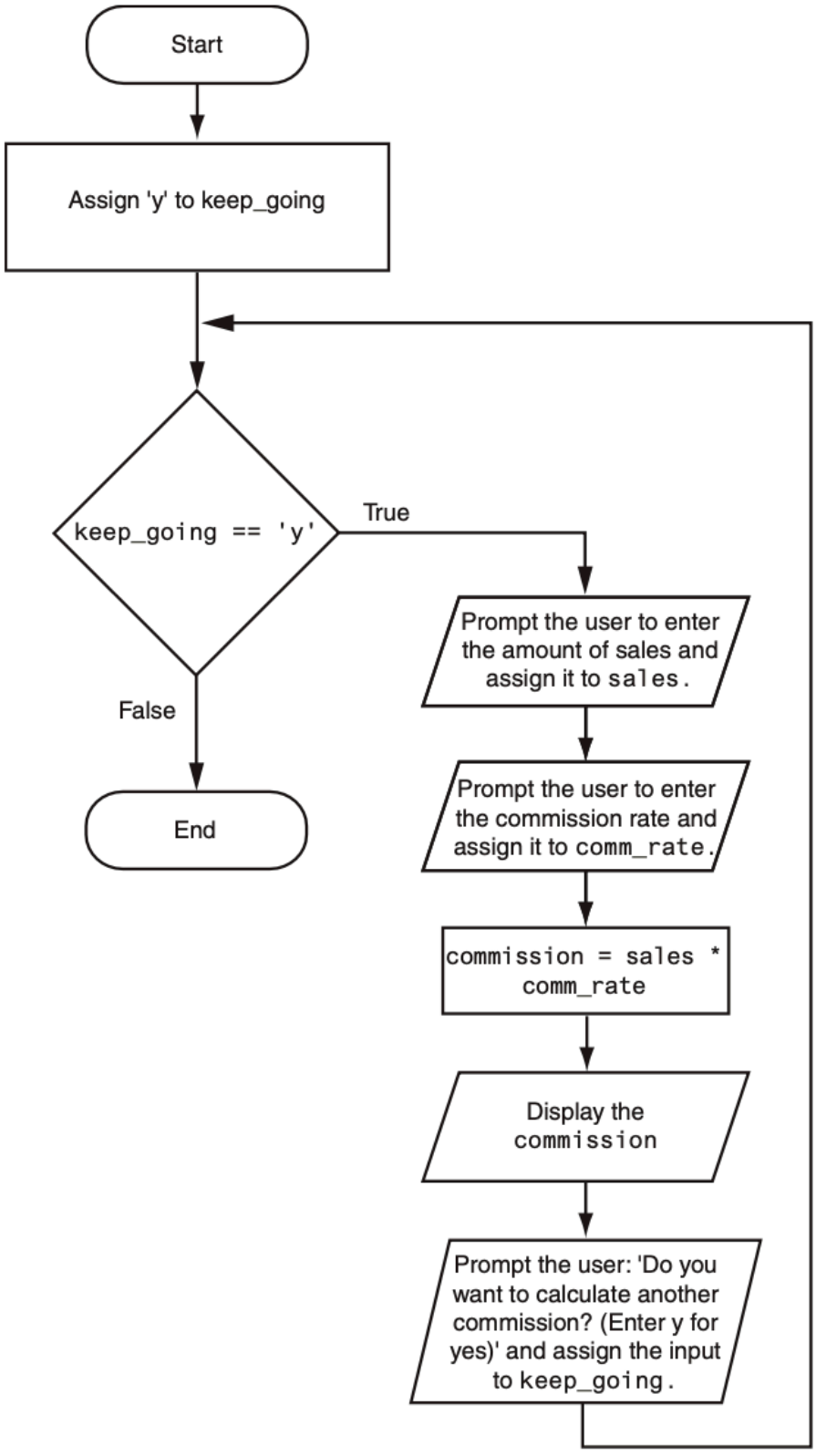
```
while condition:
  statement
  statement
  etc.
```

- The while clause begins with the word while, followed by a Boolean condition that will be evaluated as either true or false.
- A colon appears after the condition.
- Beginning at the next line is a block of statements.
- When the while loop executes, the condition is tested.
  - If the condition is true,
    - the statements that appear in the block following the while clause are executed, and the loop starts over.
  - If the condition is false,
    - the program exits the loop.



- In order to stop executing, inside the loop to make the expression keep_going == ‘y’ false.
  - “Do you want to calculate another commission (Enter y for yes).”
  - If the user enters y (and it must be a lowercase y),
    - Then the expression keep_going == ‘y’ will be true when the loop starts over.
    - This will cause the statements in the body of the loop to execute again.

- But if the user enters anything other than lowercase y,
- The expression will be false when the loop starts over, and the program will exit the loop.



76

The while Loop Is a Pretest Loop

- The while loop is known as a pretest loop,
  - Which means it tests its condition before performing an iteration.
  - Because the test is done at the beginning of the loop

```
while keep_going == 'y':
```

- The loop will perform an iteration only if the expression keep_going ==‘y’ is true.
- This means that
  - (a) the keep_going variable has to exist, and
  - (b) it has to reference the value ‘y’.

To make sure the expression is true the first time that the loop executes, we assigned the value ‘y’ to the keep_going variable

```
keep_going = 'y'
```

- By performing this step we know that the condition keep_going ==‘y’ will be true the first time the loop executes.
- This is an important characteristic of the while loop:
  - it will never execute if its condition is false to start with.
  - In some programs, this is exactly what you want.

Infinite Loops

- Something inside the loop must eventually make the test condition false.
- If a loop does not have a way of stopping, it is called an **infinite loop**.
  - An infinite loop continues to repeat until the program is interrupted.
  - Infinite loops usually occur when the programmer forgets to write code inside the loop that makes the test condition false.
  - In most circumstances, you should avoid writing infinite loops.
  - As a consequence, the loop has no way of stopping. (The only way to stop this program is to press Ctrl+C on the keyboard to interrupt it.)

```
# code_04_03_infinite.py
# This program demonstrates an infinite loop.
# Create a variable to control the loop.
keep_going = 'y'

# Warning! Infinite loop!
while keep_going == 'y':
```



```
    # Get a salesperson's sales and commission rate.
    sales = float(input('Enter the amount of sales: '))
    comm_rate = float(input('Enter the commission rate: '))

    # Calculate the commission.
    commission = sales * comm_rate
    # Display the commission.
    print('The commission is $', format(commission, ',.2f'), sep='')
```

The for Loop: A Count-Controlled Loop

A count-controlled loop iterates a specific number of times. In Python, you use the for statement to write a count-controlled loop.

- A count-controlled loop iterates a specific number of times.
- Count-controlled loops are commonly used in programs.
  - For example, Each time the loop iterates, it will prompt the user to enter the sales for one day.
  - You use the for statement to write a count-controlled loop.
- In Python, the for statement is designed to work with a sequence of data items.
- When the statement executes, it iterates once for each item in the sequence.

```
for variable in [value1, value2, etc.]:                    ①
  statement
  statement
  etc.
```

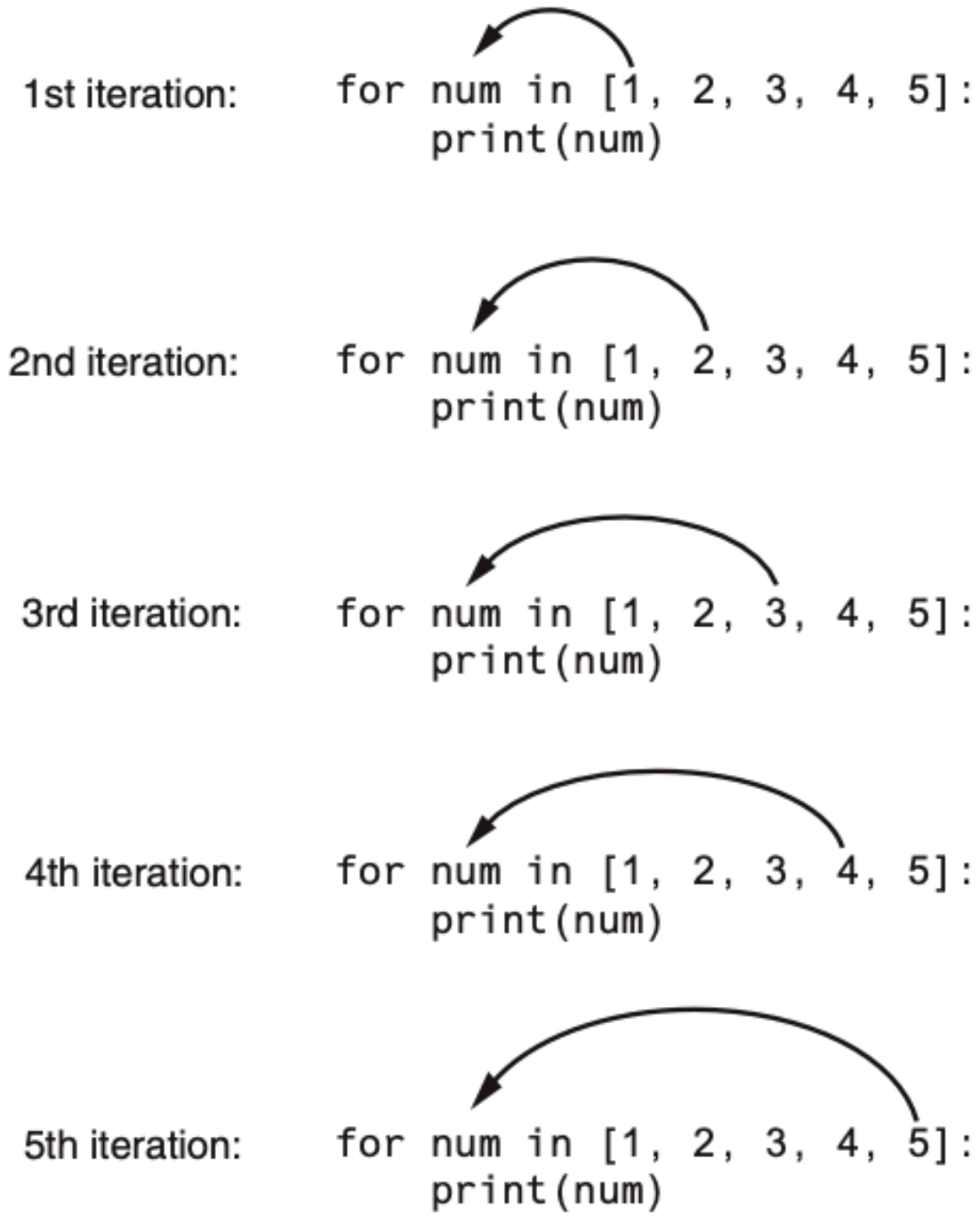
① for clause

- Format
  - In the for clause, variable is the name of a variable.
  - Inside the brackets a sequence of values appears, with a comma separating each value.
    - In Python, a comma-separated sequence of data items that are enclosed in a set of brackets is called a list
  - Beginning at the next line is a block of statements that is executed each time the loop iterates.
- The for statement executes in the following manner:
  - The variable is assigned the first value in the list,
  - Then the statements that appear in the block are executed.

  - Then, variable is assigned the next value in the list, and the statements in the block are executed again.
  - This continues until variable has been assigned the last value in the list.

- Iteration

  - The first time the for loop iterates, the num variable is assigned the value 1 and then the statement in line 6 executes (displaying the value 1).
  - The next time the loop iterates, num is assigned the value 2, and the statement in line 6 executes (displaying the value 2).
  - This process continues until num has been assigned the last value in the list.
  - Because the list contains five values, the loop will iterate five times.



- Python programmers commonly refer to the variable that is used in the for clause as the target variable because it is the target of an assignment at the beginning of each loop iteration.
- The values that appear in the list do not have to be a consecutively ordered series of numbers.

- For example, next program uses a for loop to display a list of odd numbers.
- There are five numbers in the list, so the loop iterates five times.

```
# code_04_05_simple_loop2.py
print('I will display the odd numbers 1 through 9.')
for num in [1, 3, 5, 7, 9]:
    print(num)
```

```
# code_04_06_simple_loop3.py
# This program also demonstrates a simple for
# loop that uses a list of strings.

for name in ['Winken', 'Blinken', 'Nod']:
    print(name)
```

Using the range Function with the for Loop

- Python provides a built-in function named range that simplifies the process of writing a count-controlled for loop.
- The range function creates a type of object known as an iterable.
- An iterable is an object that is similar to a list.
- It contains a sequence of values that can be iterated over with something like a loop.

```
for num in range(5):
  print(num)
```

- Notice instead of using a list of values, we call to the range function passing 5 as an argument.
- In this statement, the range function will generate an iterable sequence of integers in the range of 0 up to (but not including) 5.

```
for num in [0, 1, 2, 3, 4]:
  print(num)
```

- As you can see, the list contains five numbers, so the loop will iterate five times. Next program uses the range function with a for loop to display “Hello world” five times.

```
# code_04_07_simple_loop4.py
# This program demonstrates how the range
# function can be used with a for loop.

# Print a message five times.
for x in range(6):
    print('Hello world')
```



```
for num in range(1, 6):
    print(num)

for num in range(1, 10, 2):
    print(num)
```

- If you pass one argument to the range function that argument is used as the ending limit of the sequence of numbers.
- If you pass two arguments to the range function,
  - the first argument is used as the starting value of the sequence, and
  - the second argument is used as the ending limit.

```
for num in range(1, 5):
  print(num)

#This code will display the following:
1
2
3
4
```

- By default, the range function produces a sequence of numbers that increase by 1 for each successive number in the list.
- If you pass a third argument to the range function, that argument is used as step value.
- Instead of increasing by 1, each successive number in the sequence will increase by the step value.

```
for num in range(1, 10, 2):
  print(num)

#This code will display the following:
1
3
5
7
9
```

- In this for statement, three arguments are passed to the range function:
  - The first argument, 1, is the starting value for the sequence.
  - The second argument, 10, is the ending limit of the list. This means that the last number in the sequence will be 9.
  - The third argument, 2, is the step value. This means that 2 will be added to each successive number in the sequence.

Using the Target Variable Inside the Loop

- In a for loop, the purpose of the target variable is to reference each item in a sequence of items as the loop iterates.
- In many situations it is helpful to use the target variable in a calculation or other task within the body of the loop.
- For example, suppose you need to write a program that displays the numbers 1 through 10 and their respective squares

Number	Square
1	1
2	4
3	9
4	16
5	25
6	36
7	49
8	64
9	81
10	100

- This can be accomplished by writing a for loop that iterates over the values 1 through 10.
- During the first iteration, the target variable will be assigned the value 1,
- During the second iteration it will be assigned the value 2, and so forth.
- Because the target variable will reference the values 1 through 10 during the loop's execution, you can use it in the calculation inside the loop.

```
# code_04_08_squares.py
# This program uses a loop to display a
# table showing the numbers 1 through 10
```



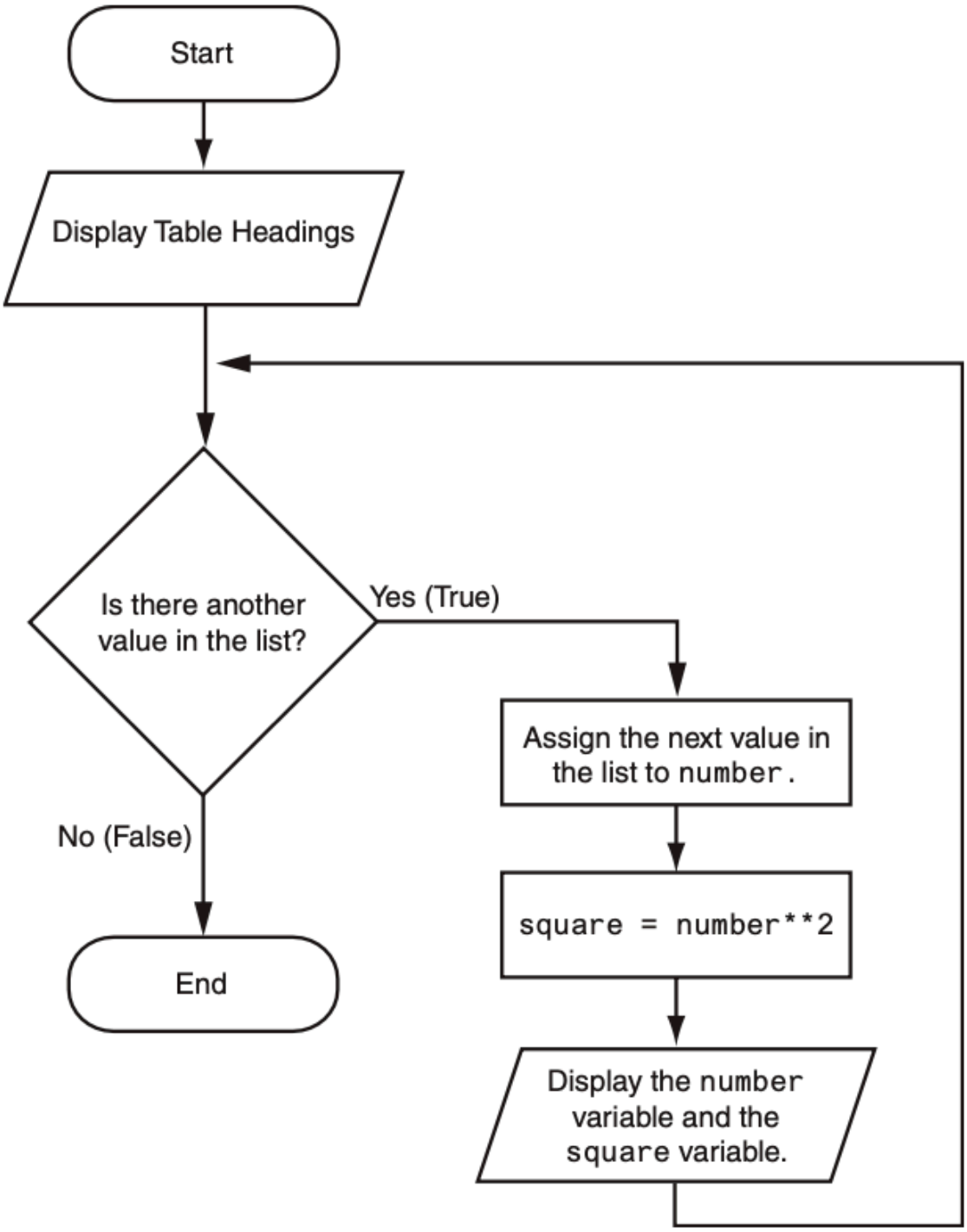
```
# and their squares.

# Print the table headings.
print('Number\tSquare')                                                    ①
print('--------------')

# Print the numbers 1 through 10
# and their squares.
for number in range(1, 11):
    square = number**2
    print(number, '\t', square)
```

① Table heading

- Notice inside the string literal, the '' escape sequence between the words Number and Square.
- The escape sequence is like pressing the Tab key
  - It causes the output cursor to move over to the next tab position.
  - This causes the space that you see between the words Number and Square in the sample output.



181

Letting the User Control the Loop Iterations

- When the code was written, the programmer knew that the loop had to iterate over the values 1 through 10.
- Sometimes, the programmer needs to let the user control the number of times that a loop iterates.
- For example, what if you want the next program to be a bit more versatile by allowing the user to specify the maximum value displayed by the loop?

```
# code_04_10_user_squares1.py
# This program uses a loop to display a
# table of numbers and their squares.

# Get the ending limit.
print('This program displays a list of numbers')
print('(starting at 1) and their squares.')
end = int(input('How high should I go? '))

# Print the table headings.
print()
print('Number\tSquare')
print('--------------')

# Print the numbers and their squares.
for number in range(1, end + 1):
    square = number**2
    print(number, '\t', square)
```

- This program asks the user to enter a value that can be used as the ending limit for the list.
- This value is assigned to the end variable in line 7.
- Then, the expression end + 1 is used in line 15 as the second argument for the range function.

Generating an Iterable Sequence that Ranges from Highest to Lowest

- In the examples you have seen so far, the range function was used to generate a sequence with numbers that go from lowest to highest.
- Alternatively, you can use the range function to generate sequences of numbers that go from highest to lowest.

- In this function call, the starting value is 10, the sequence's ending limit is 0, and the step value is -1. This expression will produce the following sequence:


```
range(10, 0, -1)

10, 9, 8, 7, 6, 5, 4, 3, 2, 1
```

- Here is an example of a for loop that prints the numbers 5 down to 1:

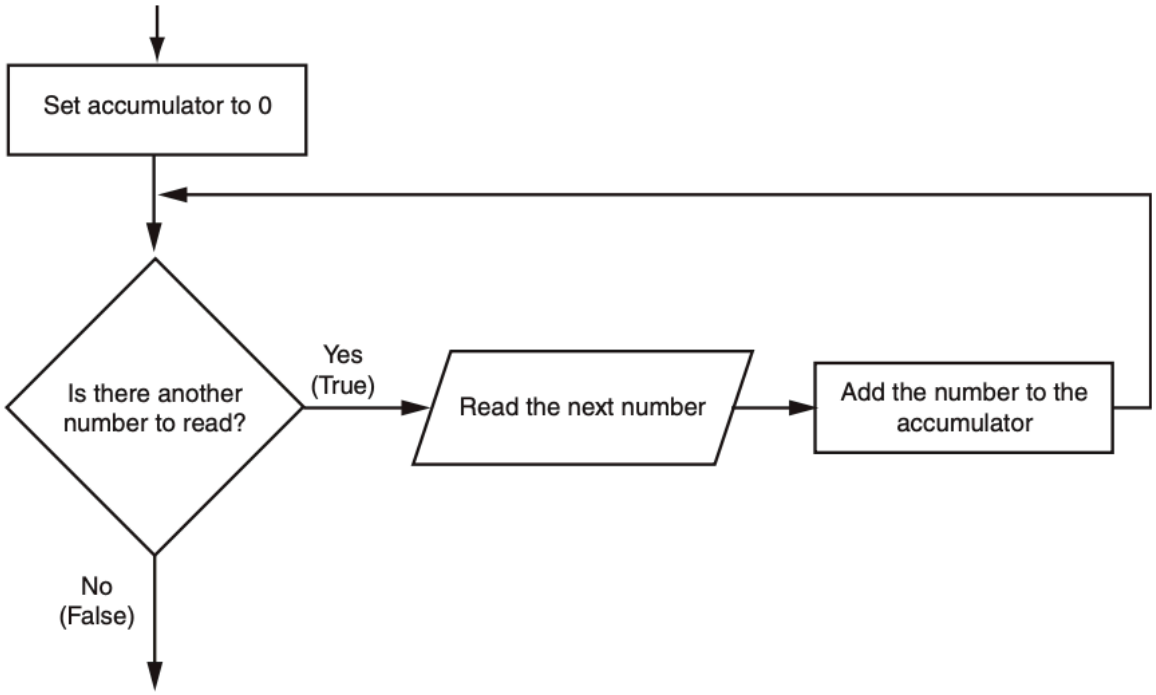
```
for num in range(5, 0, -1):
  print(num)
```

Calculating a Running Total

A running total is a sum of numbers that accumulates with each iteration of a loop. The variable used to keep the running total is called an accumulator.

- Many programming tasks require you to calculate the total of a series of numbers.
- For example, suppose you are writing a program that calculates a business's total sales for a week.
  - The program would read the sales for each day as input and calculate the total of those numbers.
  - Programs that calculate the total of a series of numbers typically use two elements:
    * A loop that reads each number in the series.
    * A variable that accumulates the total of the numbers as they are read.

The variable that is used to accumulate the total of the numbers is called an accumulator. It is often said that the loop keeps a running total because it accumulates the total as it reads each number in the series.



- When the loop finishes, the accumulator will contain the total of the numbers that were read by the loop.
- Notice the first step in the flowchart is to set the accumulator variable to 0.
- This is a critical step. Each time the loop reads a number, it adds it to the accumulator.
- If the accumulator starts with any value other than 0, it will not contain the correct total when the loop finishes.
- Let's look at a program that calculates a running total. Program allows the user to enter five numbers, and displays the total of the numbers entered.

```
# code_04_12_sum_numbers.py
# This program calculates the sum of a series
# of numbers entered by the user.

MAX = 5 # The maximum number

# Initialize an accumulator variable.
total = 0.0

# Explain what we are doing.
print('This program calculates the sum of')
print(MAX, 'numbers you will enter.')

# Get the numbers and accumulate them.
for counter in range(MAX):
```



```
    number = int(input('Enter a number: '))
    total = total + number

# Display the total of the numbers.
print('The total is', total)
```

- The total variable, created by the assignment statement in line 7, is the accumulator.
- Notice it is initialized with the value 0.0.
- The for loop, in lines 14 through 16, does the work of getting the numbers from the user and calculating their total.
- Line 15 prompts the user to enter a number then assigns the input to the number variable.
- Then, the following statement in line 16 adds number to total:

```
total = total + number
```

- After this statement executes, the value referenced by the number variable will be added to the value in the total variable.
- How this statement works.
  - First, the interpreter gets the value of the expression on the right side of the = operator, which is total + number.
  - Then, that value is assigned by the = operator to the total variable.
  - The effect of the statement is that the value of the number variable is added to the total variable.
  - When the loop finishes, the total variable will hold the sum of all the numbers that were added to it.

The Augmented Assignment Operators

- Quite often, programs have assignment statements in which the variable that is on the left side of the '=' operator also appears on the right side of the '=' operator.

```
x = x + 1
```

- On the right side of the assignment operator, 1 is added to x. The result is then assigned to x, replacing the value that x previously referenced.
- Effectively, this statement adds 1 to x.

```
total = total + number
```

- This statement assigns the value of total + number to total.
- The effect of this statement is that number is added to the value of total.

```
balance = balance - withdrawal
```

- This statement assigns the value of the expression "balance − withdrawal" to balance.

- The effect of this statement is that withdrawal is subtracted from balance.

Statement	What It Does	Value of x after the Statement
`x = x + 4`	Add 4 to x	10
`x = x - 3`	Subtracts 3 from x	3
`x = x * 10`	Multiplies x by 10	60
`x = x / 2`	Divides x by 2	3
`x = x % 4`	Assigns the remainder of x / 4 to x	2

- These types of operations are common in programming.
- For convenience, Python offers a special set of operators designed specifically for these jobs.

Operator	Example Usage	Equivalent To
`+=`	`x += 5`	`x = x + 5`
`-=`	`y -= 2`	`y = y - 2`
`*=`	`z *= 10`	`z = z * 10`
`/=`	`a /= b`	`a = a / b`
`%=`	`c %= 3`	`c = c % 3`
`//=`	`x //= 3`	`x = x // 3`
`**=`	`y **= 2`	`y = y**2`

- As you can see, the augmented assignment operators do not require the programmer to type the variable name twice.

```
total = total + number
```

could be rewritten as

```
total += number
```

Similarly, the statement

```
balance = balance - withdrawal
```

could be rewritten as

```
balance -= withdrawal
```


Sentinels

A sentinel is a special value that marks the end of a sequence of values.

- Consider this situation:
  - You are creating a program to process a series of values using a loop.
  - When designing your program, you don't yet know how many values will be entered.
  - The number of values might even change each time the program runs.
- So, what is the best way to design this loop?
  - You could ask the user after each loop iteration if there's another item, but if the sequence is long, asking repeatedly would be inconvenient.
  - Alternatively, you could ask the user to specify at the start how many values there are, but this would require extra effort if the user doesn't already know or would have to count the items.

  A better and simpler solution is usually to use a "sentinel" value. A sentinel is a special value that signals the end of a series.

- Here's how a sentinel works with an example:
  - Suppose a doctor wants a program to calculate the average weight of her patients.
  - The program repeatedly asks the user to enter each patient's weight or enter 0 to show that there are no more weights.
  - The number 0 here is the "sentinel" value, signaling the end of the data input.
  - When the user inputs 0, the program understands that no more values will be entered, so it stops the loop and calculates and displays the average.

  A sentinel must be unique enough to not be confused with regular input. In this example, using "0" works well because a patient will never weigh 0 pounds.

Input Validation Loops

Input validation is the process of inspecting data that has been input to a program, to make sure it is valid before it is used in a computation. Input validation is commonly done with a loop that iterates as long as an input variable references bad data.

- "garbage in, garbage out.", sometimes abbreviated as GIGO

  - Refers to the fact that computers can not tell the difference between good data and bad data.
  - If a user provides bad data as input to a program, the program will process that bad data and,
  - As a result, will produce bad data as output.

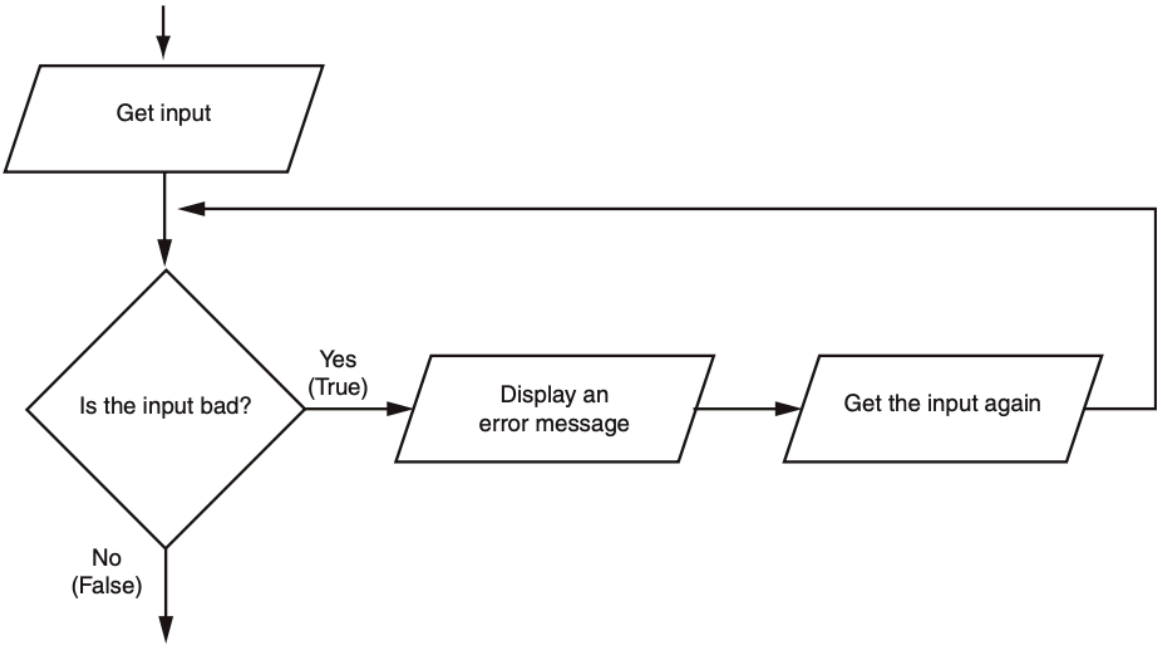
```
# code_04_14_gross_pay.py
# This program displays gross pay.
# Get the number of hours worked.
hours = int(input('Enter the hours worked this week: '))

# Get the hourly pay rate.
pay_rate = float(input('Enter the hourly pay rate: '))

# Calculate the gross pay.
gross_pay = hours * pay_rate

# Display the gross pay.
print('Gross pay: $', format(gross_pay, ',.2f'))
```

- Did you notice the mistake in the data entry example?
  - The payroll clerk mistakenly typed “400” hours instead of “40”.
  - A person getting paid for 400 hours instead of 40 would be very happy, but clearly, it’s an error.
  - The computer doesn’t know it’s incorrect—it processed the data as if it were correct.
- Can you think of other examples of incorrect input for a payroll program?
  - Negative number of hours worked
  - Pay rates that are unrealistically high or low
- You might hear stories about “computer errors”:
  - Customers charged thousands of dollars for small purchases
  - People receiving unusually large tax refunds
  - Usually, these problems aren’t caused by the computer itself, but by incorrect data entry.
- The accuracy of a program’s output depends on correct input:
  - Always carefully check (validate) input before processing it.
  - If input is invalid, your program should:
    - Display a clear error message
    - Prompt the user to re-enter valid input
  - This approach is called “input validation”.
- Common steps in the input validation process (see Figure 4-7):
  - User inputs data.
  - The program checks data validity.
  - If input is wrong, the program displays an error message and asks for input again.
  - Repeat these steps until valid data is entered.



- Notice the flowchart reads input in two places:
  - First just before the loop, and then inside the loop.
  - The first input operation—just before the loop—is called a **priming read**, and its purpose is to get the first input value that will be tested by the validation loop.
- If that value is invalid, the loop will perform subsequent input operations.

- Suppose you are designing a program that reads a test score and you want to make sure the user does not enter a value less than 0.
- The following code shows how you can use an input validation loop to reject any input value that is less than 0.

```
# Get a test score.
score = int(input('Enter a test score: '))
# Make sure it is not less than 0.
while score < 0:
  print('ERROR: The score cannot be negative.')
  score = int(input('Enter the correct score: '))
```

- The code first asks the user to enter a test score (known as the priming read).
- Then, a while loop checks if the score is valid.
  - Remember, a while loop always checks a condition before it runs.

- If the initial input is valid (score is not less than 0), the loop doesn't run at all.
- If the initial input is invalid (score less than 0), the loop performs these actions:
  - Shows an error message.
  - Prompts the user to enter a correct test score.
  - Repeats these steps until the user enters valid input.

> **Note**
>
> An input validation loop is sometimes called an error trap or an error handler.

- This code rejects only negative test scores. What if you also want to reject any test scores that are greater than 100? You can modify the input validation loop so it uses a compound Boolean expression, as shown next.

```
# Get a test score.
score = int(input('Enter a test score: '))
# Make sure it is not less than 0 or greater than 100.
while score < 0 or score > 100:
  print('ERROR: The score cannot be negative')
  print('or greater than 100.')
  score = int(input('Enter the correct score: '))
```

- The loop in this code determines whether score is less than 0 or greater than 100.
- If either is true, an error message is displayed and the user is prompted to enter a correct score.

Nested Loops

A loop that is inside another loop is called a nested loop.

- A nested loop is a loop placed inside another loop.
- A clock is an example of how nested loops work:
  - The second hand, minute hand, and hour hand each rotate in their own cycles.
  - The second hand rotates 60 times for the minute hand to complete one rotation.
  - The minute hand rotates 12 times for the hour hand to complete one rotation.
  - This means the second hand completes 720 rotations for each full rotation of the hour hand.

```
It displays the seconds from 0 to 59:
for seconds in range(60):
  print(seconds)
```


- We can add a minutes variable and nest the loop above inside another loop that cycles through 60 minutes:

```
for minutes in range(60):
  for seconds in range(60):
    print(minutes, ':', seconds)
```

- To make the simulated clock complete, another variable and loop can be added to count the hours:

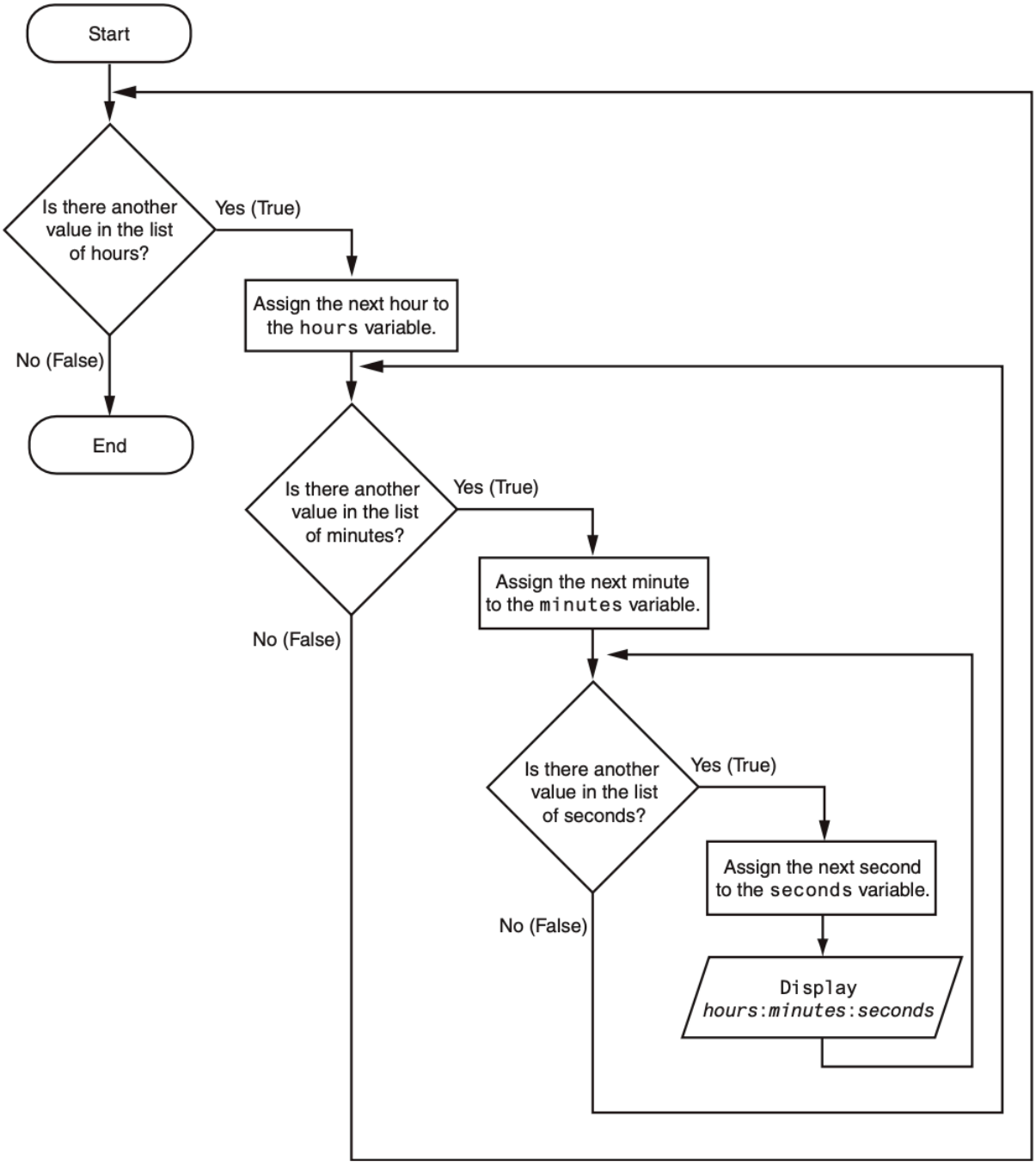
```
for hours in range(24):
  for minutes in range(60):
    for seconds in range(60):
      print(hours, ':', minutes, ':', seconds)

#This code's output would be:
0 : 0 : 0
0 : 0 : 1
0 : 0 : 2

#(The program will count through each second of 24 hours.)

23 : 59 : 59
```

- The innermost loop will iterate 60 times for each iteration of the middle loop.
- The middle loop will iterate 60 times for each iteration of the outermost loop.
- When the outermost loop has iterated 24 times, the middle loop will have iterated 1,440 times and the innermost loop will have iterated 86,400 times!



- The clock example highlights some key ideas about nested loops:
  - An inner loop fully completes all its iterations for every single iteration of the outer loop.
  - Inner loops finish their cycles faster than outer loops.
  - To get the total number of iterations, multiply the number of iterations of each loop together.
- Here's another example (Program 4-17) showing nested loops:

  - A teacher needs a program to find the average of test scores for a group of students.
  - First, the program asks for the number of students.
  - Next, it asks for how many test scores each student will have.
  - An outer loop runs once for each student.
  - A nested (inner) loop runs once for each test score per student.

Turtle Graphics: Using Loops to Draw Designs

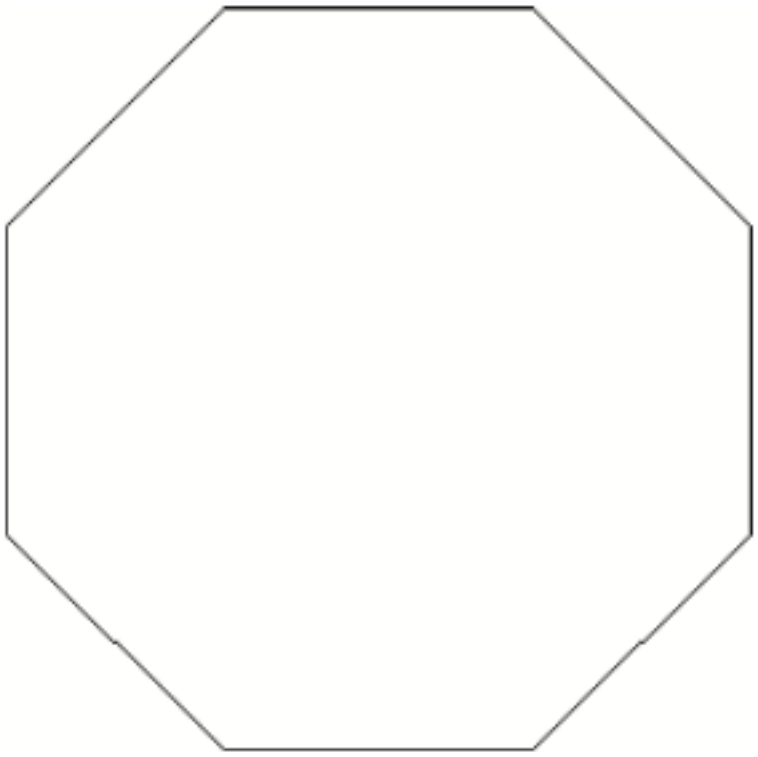
You can use loops to draw graphics that range in complexity from simple shapes to elaborate designs.

- You can use loops with the turtle to draw both simple shapes and elaborate designs.
- For example, the following for loop iterates four times to draw a square that is 100 pixels wide:

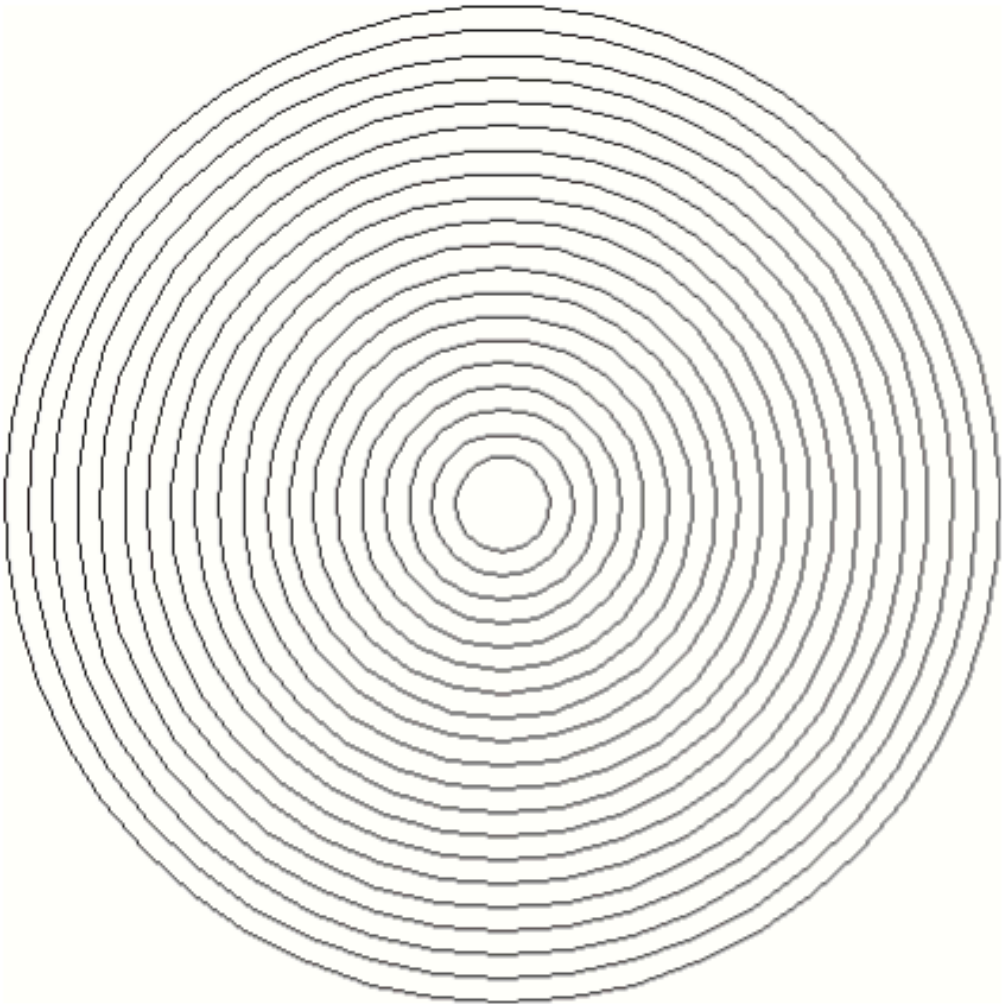
```
for x in range(4):
  turtle.forward(100)
  turtle.right(90)
```

- The following code shows another example. This for loop iterates eight times to draw the octagon

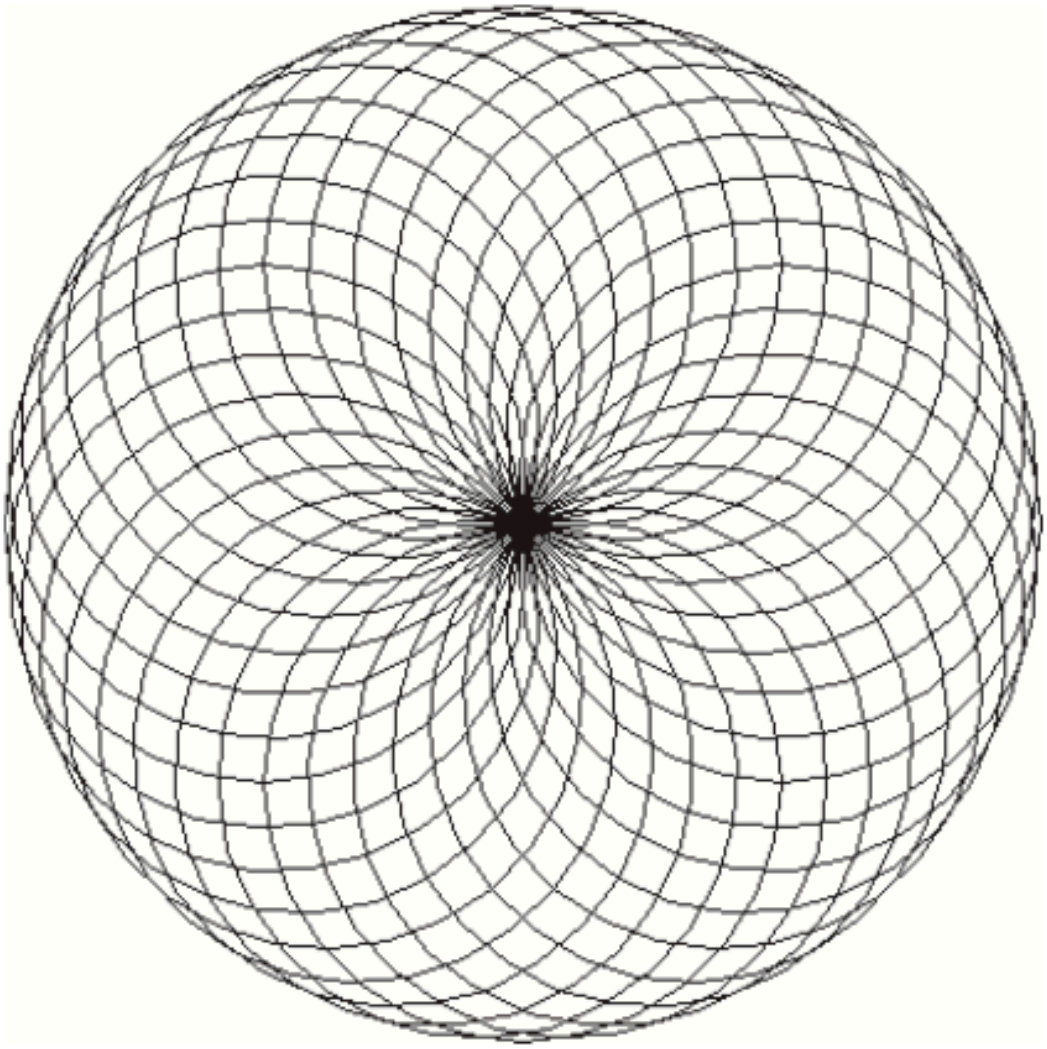
```
for x in range(8):
  turtle.forward(100)
  turtle.right(45)
```

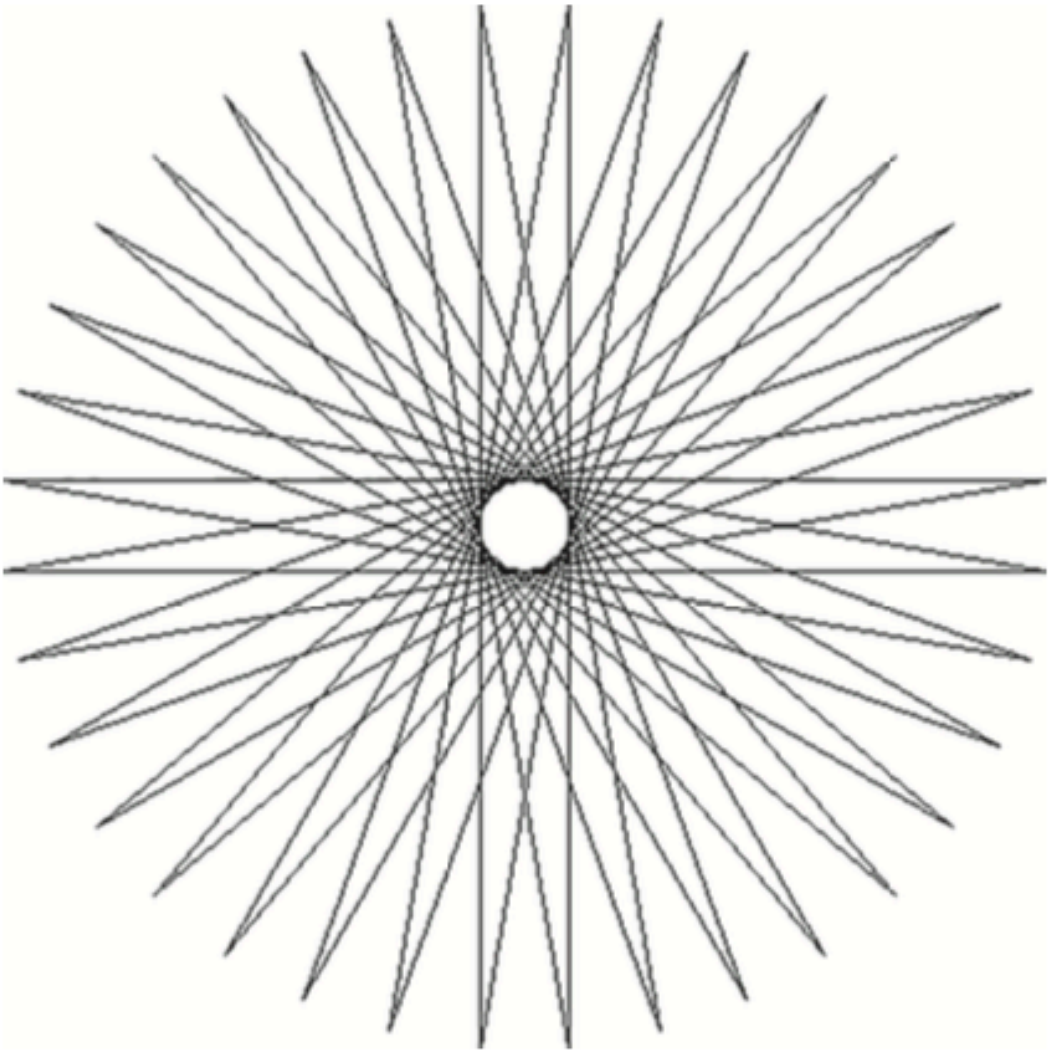
- You can create a lot of interesting designs with the turtle by repeatedly drawing a simple shape, with the turtle tilted at a slightly different angle each time it draws the shape.



- For example, drawing 36 circles with a loop.
  - After each circle was drawn, the turtle was tilted left by 10 degrees.



375



House Keeping

Announcement

Assignment

Assignment1: Assignment Title

- **What to do**
  - A

  - B

- **Format and Requirement**
  - PDF format, < ?? pages
  - file name should be include your student id and name
    - stuID_name_title.pdf (e.g. 1111111_ChungilChae_SelfIntroduction.pdf)
  - APA, Double-Space, No title page, 12 points, Reference section if necessary
  - Chinese name in English, English name, Student ID, Class name and section (e.g. GE2021, W09; MGS3001, W01)
- due date
  - by Date(DAY) 11:59PM
  - NO LATE SUBMISSION ALLOWED!!!!

Reference

